Day 3 — Object-Oriented Programming (OOPs)

- 1.**Classes and objects:-**
- **Class:** A blueprint or template used to create objects.
  - It defines the structure (fields/attributes) and behavior (methods) that all objects created from it will possess.
  - A class is a logical entity; it does not occupy memory until an object of that class is instantiated.

- **Object:** An instance of a class that has data (properties) and behavior (methods).
  - An **object in Java** is a fundamental building block of Object-Oriented Programming (OOP) that represents a real-world entity. It is an instance of a class [1, 2] that possesses its own **state**, **behavior**, and **identity**.
  - When a program runs, objects are created dynamically in the heap memory using the `new` keyword.

- **Constructors**:-
- Java, a **constructor** is a special block of code that is automatically invoked when an instance (object) of a class is created. Its primary purpose is to **initialize the newly created object** by setting initial values for its attributes or executing any necessary setup steps.

  - **Types of Constructors**

Constructor Type	Description
Default Constructor	Inserted automatically by the Java compiler if no constructor is explicitly written. It initializes instance variables with default values (e.g., `0` for integers, `null` for objects).
No-Arg Constructor	Explicitly defined by the programmer but accepts no arguments. Used to set custom default values.
Parameterized Constructon	Accept Accepts arguments to initialize instance variables with specific user defined values at the time of creation.

1.Encapsulation:-

Encapsulation in Java is the mechanism of wrapping data (variables) and code (methods) together as a single unit. It is one of the four core pillars of Object-Oriented Programming (OOP) and is primarily used to achieve **data hiding**, ensuring that an object's internal state is protected from unauthorized direct access and modification.

2.Inheritance:-

Inheritance in Java is a fundamental Object-Oriented Programming (OOP) mechanism that allows one class (the child/subclass) to acquire the properties (fields) and behaviors (methods) of another class (the parent/superclass).

It promotes **code reusability** and establishes an **"IS-A" relationship** (e.g., a Car *is-a* Vehicle).

Types of Inheritance Supported in Java

Java supports some types of inheritance directly through classes, while others require interfaces.

Inheritance Type	Supported via Classes?	Description
Single	**Yes**	One child class inherits from one parent class (A → B).
Multilevel	**Yes**	A child class acts as a parent for another child class (A → B → C).
Hierarchical	**Yes**	Multiple child classes inherit from a single parent class (A → B and A → C).
Multiple	**✕ No**	One child class inheriting from multiple parent classes. **Achieved only via Interfaces** to avoid ambiguity (the "Diamond Problem").
Hybrid	**✕ No**	A mix of two or more inheritance types. **Achieved only via Interfaces**.

3.Polymorphism:-

in Java is the ability of an object to take on many forms. It allows a single interface or superclass reference to represent different underlying data types (subclasses) and execute different behaviors based on the actual object being referenced.

Java supports two main types of polymorphism: **Compile-time (Static)** and **Runtime (Dynamic)**.

1. Compile-Time Polymorphism (Method Overloading)

This occurs when multiple methods in the **same class** share the same name but have **different parameter lists** (different number, type, or order of arguments).

2. Runtime Polymorphism (Method Overriding)

This occurs when a subclass provides a **specific implementation** of a method that is already defined in its superclass. Java determines which method to execute at runtime based on the **actual object type**, not the reference type. This is driven by **Upcasting**.

4.Abstraction:-

in Java is the process of hiding internal implementation details and showing only the essential features of an object to the user. It reduces complexity by allowing you to focus on *what* an object does rather than *how* it does it.

You can achieve abstraction in Java using two mechanisms: **Abstract Classes** (0% to 100% abstraction) and **Interfaces** (100% abstraction).

. Abstract Classes

An abstract class is a class declared with the `abstract` keyword. It cannot be instantiated directly and may contain both abstract methods (methods without a body) and concrete methods (methods with a body).

. Interfaces

An interface is a blueprint of a class that contains only abstract methods (by default) and static constants. It represents an absolute contract that implementing classes must fulfill.

Access modifiers:-

Java access specifiers (also called access modifiers) regulate the visibility and accessibility of classes, constructors, fields, and methods across different packages and classes.

Access modifiers in Java are used to control the visibility and accessibility of classes, methods, and variables. They help enforce encapsulation by restricting access to different parts of a program. Java provides four types of access modifiers to define scope and protection levels.

- **Public modifier:** Accessible from anywhere in the program
- **Protected modifier:** Accessible within the same package and by subclasses
- **Private modifier:** Accessible only within the same class
- **Default modifier:** Accessible only within the same package

Methods and method overloading:-

In Java, a **method** is a block of code designed to perform a specific task, which only runs when it is called. Method overloading is a feature that allows a class to have more than one method with the same name, provided their parameter lists are different.

---

1. Java Methods

Methods are used to reuse code, structure programs cleanly, and execute specific logic.

Syntax:

java

```
modifier returnType methodName(parameterList) {
    // Method body
    return value; // if returnType is not void
}
```

Use code with caution.

- **Modifier:** Defines the access type (e.g., `public`, `private`, `static`).
- **Return Type:** The data type of the value the method returns (use `void` if it returns nothing).
- **Method Name:** The unique identifier used to call the method.
- **Parameter List:** Input variables passed into the method (optional).

- ## Method Overloading
- Method overloading allows you to use the **same method name** for multiple operations, as long as the methods differ in their **parameter signatures**.
- It is a form of **Compile-time Polymorphism** (or Static Binding), meaning the compiler determines which method to execute based on the arguments passed at compile time.

TASK:-

Create Student and Employee classes.

```
class Student {
    int rollNo;
    String name;

    void display Student() {
        System.out.println("Student Details:");
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }
}

class Employee {
    int empId;
    String empName;
double salary;

    void display Employee() {
        System.out.println("\nEmployee Details:");
        System.out.println("Employee ID: " + empId);
        System.out.println("Name: " + empName);
        System.out.println("Salary: " + salary);
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        // Creating Student object
        Student s1 = new Student();
        s1.rollNo = 101;

s1.name = "Rahul";

        // Creating Employee object
        Employee e1 = new Employee();
        e1.empId = 201;
        e1.empName = "Priya";
        e1.salary = 25000.0;

        // Displaying details
        s1.displayStudent();
        e1.displayEmployee();
    }
}
```

Output:-

```
Student Details:
Roll No: 101
Name: Rahul

Employee Details:
Employee ID: 201
Name: Priya
Salary: 25000.0
```

Create Student and Employee classes.

- Add properties and methods.
- Demonstrate constructors, inheritance, and encapsulation.

```
Class Person {
    // Encapsulation: private properties
    private String name;
    private int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getters and setters
```

```
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
    }

    // Method
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Inheritance: Student inherits Person
class Student extends Person {
    private int rollNo;
    private String course;

    // Constructor
    public Student(String name, int age,
                   int rollNo, String course) {
        super(name, age);
        this.rollNo = rollNo;
        this.course = course;
    }

    public void displayStudent() {
        displayInfo();
        System.out.println("Roll Number: " + rollNo);
        System.out.println("Course: " + course);
    }

    public void study() {
System.out.println(getName() + " is studying.");
    }
}

// Inheritance: Employee inherits Person
class Employee extends Person {
    private int employeeId;
    private double salary;
```

```
    // Constructor
    public Employee(String name, int age,
                    int employeeId, double salary) {
        super(name, age);
        this.employeeId = employeeId;
        this.salary = salary;
    }

    public void displayEmployee() {
        displayInfo();
        System.out.println("Employee ID: " + employeeId);
        this.salary = salary;
    }

    public void displayEmployee() {
        displayInfo();
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Salary: " + salary);
    }
}

    public void work() {
        System.out.println(getName() + " is working.");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Student s = new Student(
            "Rahul", 20, 101, "Computer Science"
        );

        Employee e = new Employee(
            "Priya", 30, 501, 45000.0
        );
System.out.println("Student Details:");
        s.displayStudent();
        s.study();

        System.out.println("\nEmployee Details:");
        e.displayEmployee();
        e.work();

        // Encapsulation using a setter
        s.setAge(21);
        System.out.println(
            "\nUpdated Student Age: " + s.getAge()
        );
    }
}
```

```
OUTPUT:-
Student Details:
Name: Rahul
Age: 20
Roll Number: 101
Course: Computer Science
Rahul is studying.

Employee Details:
Name: Priya
Age: 30
Employee ID: 501
Salary: 45000.0
Priya is working.

Updated Student Age: 21
```

Concepts demonstrated

1. **Classes and objects:** `Student` and `Employee` are classes. Objects are created using `new Student(...)` and `new Employee(...)`.
2. **Properties:** Variables such as `name`, `age`, `rollNo`, `course`, `employeeId`, and `salary` store object data.
3. **Methods:** `study()`, `work()`, `displayStudent()`, and `displayEmployee()` define object behavior.
4. **Constructors:** Constructors initialize object properties when an object is created. `super(name, age)` calls the parent class constructor.
5. **Inheritance:** Both `Student` and `Employee` extend the `Person` class, reusing its properties and methods.
6. **Encapsulation:** The properties are declared `private`. Public getter and setter methods provide controlled access to them.
   Conclusion: This program demonstrates the main object-oriented programming concepts in Java: classes, objects, properties, methods, constructors, inheritance, and encapsulation.